Ecole Polytechnique de Thies

Département du Génie Informatique et Télécommunications
Master en Ingénierie Logicielle et Intelligence Artificielle
2025/2026

Dr Samba SIDIBE

Regression linéaire, equation normale, vectorisation

Exercice 1: Regression linéaire, equation normale

La régression linéaire est un algorithme d'apprentissage supervisé de type régression. Les algorithmes de régression permettent de prédire des valeurs continues à partir des variables prédictives. Le principe est de trouver une droite qui se rapproche le plus possible d'un ensemble de points (points d'entraînement).

Considérons l'équation de prédiction d'un modèle de régression linéaire suivante:

$$\hat{y} = h_\theta(x) = \theta^T . x = \theta_0 + \theta_1 x_1 + \ldots + \theta_n x_n \tag{1,1}$$

où $\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$ est le vecteur des paramètres du modèle, θ^T transposé de θ, x vecteur des valeurs d'une observation, $\theta^T . x$ est le produit scalaire de θ^T et x et h_θ est la fonction hypothèse utilisant les paramètres θ.

Pour la suite nous allons considerer une fonction affine univariée eq[1.2] puis y ajouter du bruit et determiner le model de regression

$$f(x) = 4 + 3x \tag{1,2}$$

1. Generation des donnees d'entrainement

le module random contient une fonction $random()$ permettant de generer un nombre aleatoire dans l'intervalle $[0, 1[$

1.1. Generer un nombre $xMin$ aleatoire compris entre -10 et 2 qui sera notre x minimum et un nombre $deltaX$ entre 10 et 20 qui sera la distance maximale entre deux x. On a $xMax = xMin + deltaX$

```
import random

#generer de maniere aleatoire un entier xMin dans [-10, 2]
#xMin= generer xMin

#generer deltaX la largeur de l'intervalle compris entre 10 et 20
#deltaX=code pour generer deltax

#Deduction de xMax
#xMax=xMin+ deltaX

#affichage variables xMin, deltaX, xMax
#print(f"xMin = {xMin}")
#print(f"deltaX = {deltaX}")
#print(f"xMax = {xMax}")
```

1-2 Vecteur x d'entrainement

La fonction $np.random.rand(n, m)$ de la librairie numpy permet de generer une matrice aleatoire de n lignes et m colonnes. (Voir la documentation numpy)

Generer un vecteur aleatoire x de dimension 20 dont les elements x_i sont entre xMin et xMax

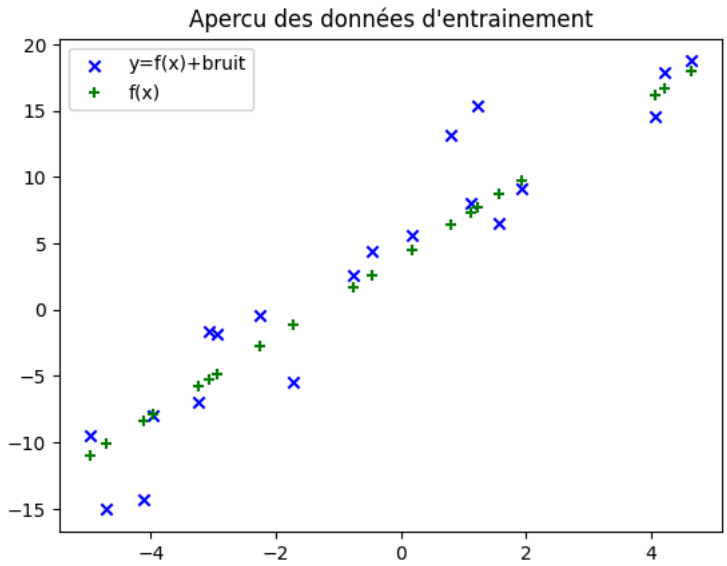
```
#générer x  avec des elements appartenant a l'intervalle [xMin, xMax]
import numpy as np

#x = generer x

#affichage de X
#print(F"X = {x}")
#affichage de la dimension de x
#print("shape",x.shape)
```

1.3. Calculer F=f(X) f etant la fonction definie en (1,2)

```
#calculer F
#F=calcul de f(x)
#affichage de la valeur de f
#print(F"F={F}")
```

1.4. Pour determiner Y on va prendre F et y ajouter du bruit gaussien. Puis on va afficher y et f(x) pour visualiser les données.

Le resultat donne une figure qui ressemble à l'image suivante




```
import matplotlib.pyplot as plt

#ajout du bruit a F
#Y = F+np.random.normal(0,3,len(x))

#Visualisation des donnees
#plt.scatter(x, Y, marker='x', color='b', label='Y')
#plt.scatter(x, F, marker='+', color='g', label='F')
#plt.title("Apercu des données d'entrainement")
#plt.legend(['y=f(x)+bruit', 'f(x)'])
#plt.show()
```

2. Regression lineaire

2.1. L'équation de régression lorsque les entrées sont composées d'une seule valeur x est donnée par

$$\widehat{y}_i = \theta_0 + \theta_1 x_1 = [1, x_1] \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$$

posons X la matrice 2 deux colonnes dont les elements de la premiere colonne ont comme valeur 1 et x_i l'element de la deuxieme colonne à la ligne *i*, $\widehat{Y}$ le vecteur constitue par les $\widehat{y}_i$

On a alors

$\widehat{Y} = h_\theta(X) = X\theta$

l'erreur quadratique moyenne peux se calculer par :

$E = \sum_{i=1}^{N} (y - y_i)^2$

Calculer les erreurs quadratiques E_1, E_2, E_3 pour θ egale aux valeurs successives suivantes: $theta1 = \begin{bmatrix} 3 \\ 7/2 \end{bmatrix}$, $theta2 = \begin{bmatrix} 5 \\ 5/2 \end{bmatrix}$ et $theta3 \begin{bmatrix} 4 \\ 3 \end{bmatrix}$

```
#Calculer X,  la premiere colonne ayant des valeurs egale à 1 et
# la deuxieme colonne etant les x_i vous pouvez utiliser les fonctions numpy
# np.ones(var) et np.c_[a,b](voir documentation)

#theta1=....
#theta2=...
#theta3=...

#Y1=calculer y1 a partir de theta1
```

```
#E1=...calculer l'erreur quadratique

#Y2=...
#E2=...

#Y3=...
#E3=...

#affichage des erreurs quadratique
#print(F"E1 {E1}")
#print(F"E2 {E2}")
#print(F"E3 {E3}")
```

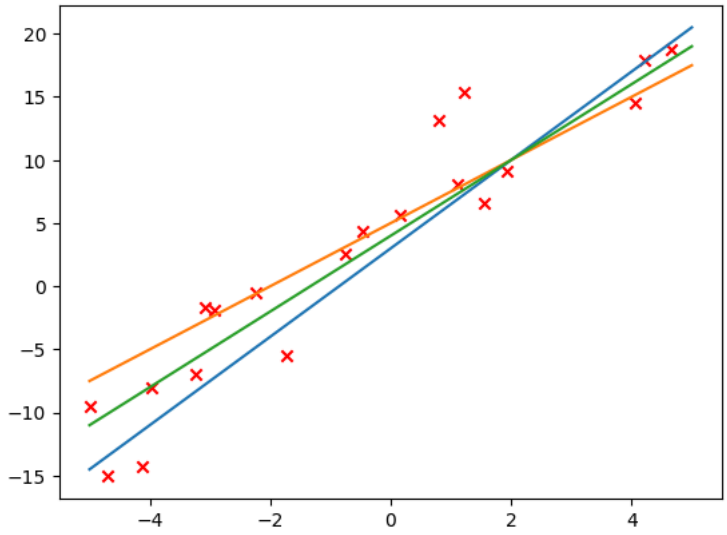
2.2. Parmis θ_1, θ_2 et θ_3 quel est le meilleur model

2.3 Creer une figure y placer les points (x_i,y_i)

2.4 Pour chaque model θ predire la valeur yMin de xMin et la valeur yMax de xMax

2.5 Pour chaque θ tracer la prediction des x qui sont sur l'intervalle delimitee par les points (xMin, yMin) et (xMax, yMax) (c'est le segment delimité par les points (xMin, yMin) et (xMax, yMax).)

Vous aurez une figure qui ressemble à la figure ci dessous




```
#afficher
#affichage des points x_i,y_i
#plt.scatter(x, Y, marker='x', color='r', label='Y')


#afficher des predictions pour les models theta1, theta2 et theta3
#plt.plot() pour theta1
#plt.plot() pour theta2
#plt.plot() pour theta3

plt.show()
```

3. Solution optimale : Equation normale

La solution analytique θ qui minimise la fonction coût est donnée par l'équation normale suivante:

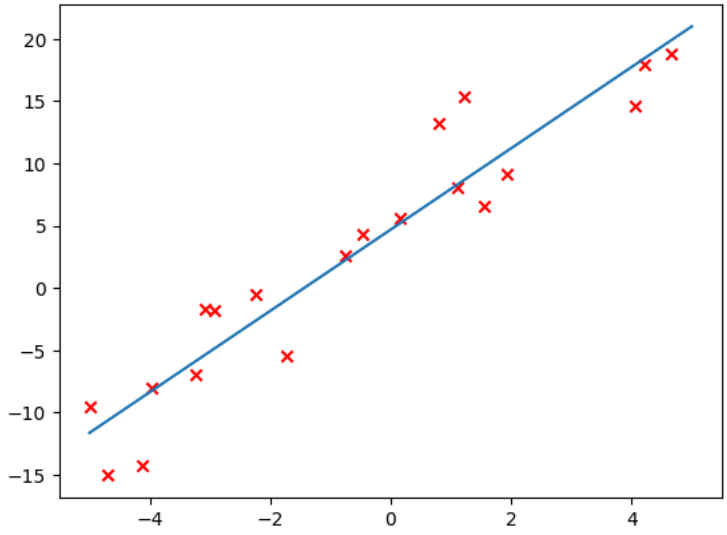
$$\widehat{\theta} = (X^T.X)^{-1}.X^T.y$$

3.1. Calculer $\widehat{\theta}$

3.2. Placer les points (x_i, y_i)

3.3. En utilisant la solution analytique, tracer la prediction des x qui sont sur l'intervalle delimitee par les points (xMin, yMin) et (xMax, yMax) (c'est le segment delimité par les points (xMin, yMin) et (xMax, yMax).)

le resultat resultat ressemblera a la figure suivante




```
# Calcul de theta chapeau thetaBest
# vous pouvez utiliser les fonction numpy suivantes np.linalg.inv
# X.T pour calculer le transposé d'une matrice X
# X.dot(Y) pour calculer le produit de deux matrices

#thetaBest=calcule thetaBest
#print("thetaBest",thetaBest)

#placement des points x_i, yi
#plt.scatter(x, Y, marker='x', color='r', label='Y')

#tracer les predictions des points x sur [xMin, xMax]
#plt.plot(.....)
plt.show()
```

Exercice 2 : importance de la vectorisation

Le produit scalaire d'un vecteur-ligne x^T par un vecteur-colonne y est donné par la formule suivante :

$$x^T y = \begin{bmatrix} x_1 x_2 \dots x_n \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n = \sum_{i=1}^{n} x_i y_i$$

Le produit de la matrice A (n × m) par la matrice B (m × p) est la matrice C (n × p) telle que l'élément C_{ij} (ligne i, colonne j) est égal au produit scalaire de la ligne i de la matrice A par la colonne j de la matrice B

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

1. Creer une fonction permettant de calculer le produit de deux matrices en utilisant des boucles

```
#produit de matrice utilisant une boucle
def product_loop(A, B):
    ##
    ##
    resultat=0
    return resultat
```

2. Creer une fonction permettant de calculer le produit de deux matrice en utilisant la fonction dot de numpy

```
#produit de matrice utilisant une vectorisation numpy
def product_numpy(A, B):
    ##
    ##
    resultat=0
    return resultat
```

3. Calculer le temps de calcule de produit de matrices carrees dont les dimension sont successivement 10, 100, 1000, .., 10000000 vous pouvez utiliser le module $time$ de python pour determiner les temps de calcul. Les matrices peuvent etre generees de maniere aleatoire avec numpy

```
### comparer les temps de calcul pour des matrices carrées de taille 10, 100, 1000, .., 10000000
```

4. Tracer sur une meme figure les courbes illustrant le temps de calcul en fonction de la taille de la matrice

```
#tracer les courbes
```